

Fiche de révision — Machine Learning

Concours ANP — Cadre Data Scientist

Table des matières

1	Concepts ML de base	2
1.1	AI vs ML vs Deep Learning	2
1.2	Apprentissage supervisé vs non supervisé	2
1.3	Classification vs régression vs clustering	2
1.4	Training, testing, validation	2
1.5	Features et target	3
2	Algorithmes	3
2.1	Linear Regression	3
2.2	Logistic Regression	4
2.3	Decision Tree	5
2.4	Random Forest	7
2.5	XGBoost	8
2.6	K-Means	8
2.7	PCA	8
3	Évaluation ML	8
3.1	Classification — Confusion Matrix	8
3.2	Classification — Métriques	8
3.3	Régression — Métriques	8
3.4	Questions types	8
4	ML avancé (Jour 3)	8
4.1	Preprocessing	8
4.2	Feature engineering et feature selection	8
4.3	Data leakage	8
4.4	Imbalanced dataset et SMOTE	8
4.5	Cross-validation	8
4.6	Hyperparameter tuning	8
4.7	Gradient Boosting et Ensemble Learning	8
4.8	Questions ouvertes	8

1 Concepts ML de base

1.1 AI vs ML vs Deep Learning

- **Intelligence Artificielle (AI)** : le domaine le plus large — toute technique qui permet à une machine d'imiter un comportement humain intelligent (raisonnement, perception, décision). Inclut des règles écrites à la main (systèmes experts), pas forcément de l'apprentissage.
- **Machine Learning (ML)** : sous-ensemble de l'AI où la machine *apprend* un modèle à partir de données, au lieu de suivre des règles écrites explicitement. On lui montre des exemples, elle en déduit un pattern généralisable.
- **Deep Learning (DL)** : sous-ensemble du ML basé sur des réseaux de neurones à plusieurs couches (*deep* = profond). Excelle quand il y a beaucoup de données non structurées (images, texte, son) et peu de features déjà préparées à la main.

Hiérarchie à retenir : $AI \supset ML \supset \text{Deep Learning}$.

Idée clé à savoir formuler : le ML classique (Random Forest, XGBoost...) reste souvent préférable sur des données tabulaires (comme des données économiques ou portuaires) car plus interprétable et moins gourmand en données/calcul que le Deep Learning.

1.2 Apprentissage supervisé vs non supervisé

- **Supervised learning (apprentissage supervisé)** : on dispose de données déjà étiquetées, c'est-à-dire qu'on connaît la réponse attendue (*target*) pour chaque exemple historique. Le modèle apprend la relation entre les features et le target pour prédire ce target sur de nouvelles données.
 - Exemple : prédire si un navire sera en retard (oui/non) à partir de son historique.
- **Unsupervised learning (apprentissage non supervisé)** : les données n'ont *pas* de target connu. Le modèle cherche uniquement une structure ou des regroupements cachés dans les données.
 - Exemple : regrouper des navires ou des clients par profil similaire sans savoir à l'avance quels groupes existent.

1.3 Classification vs régression vs clustering

- **Classification** (supervisé) : le target est une *catégorie* discrète (ex. retard / pas de retard, fraude / pas fraude).
- **Régression** (supervisé) : le target est une valeur *continue* (ex. temps d'attente en heures, montant d'une facture).
- **Clustering** (non supervisé) : regrouper des observations similaires entre elles, sans target prédéfini (ex. segmentation de clients ou de navires).

1.4 Training, testing, validation

- **Training set (entraînement)** : les données sur lesquelles le modèle apprend ses paramètres.
- **Validation set** : sert à régler les hyperparamètres et comparer plusieurs modèles *sans* toucher aux données de test, pour éviter de biaiser l'évaluation finale.
- **Test set** : données jamais vues par le modèle pendant l'entraînement, utilisées une seule fois à la fin pour mesurer la performance réelle attendue en production.

Pourquoi séparer les trois : si on évalue le modèle sur les données qui ont servi à l'entraîner, on surestime sa performance réelle (voir *overfitting* plus loin).

1.5 Features et target

- **Features (variables explicatives)** : les colonnes/informations d'entrée utilisées par le modèle pour faire une prédiction (ex. tirant d'eau du navire, type de marchandise, heure d'arrivée).
- **Target (variable cible)** : ce qu'on cherche à prédire (ex. le temps d'attente du navire).

Question type à savoir répondre à l'oral :

« Quelle différence entre classification et régression, avec un exemple portuaire ? »

2 Algorithmes

2.1 Linear Regression

Principe : trouve la droite (ou l'hyperplan, avec plusieurs features) qui passe "au plus près" de tous les points observés, en minimisant la somme des écarts au carré entre la valeur prédite et la valeur réelle. Le modèle prédit une valeur *continue* comme une combinaison linéaire pondérée des features : $y = \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_0$.

Formalisation mathématique — du simple au général

Étape 1 — l'équation avec 2 features seulement. Prenons le cas le plus simple, avec deux features x_1 et x_2 :

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

Exemple concret : prédire le temps d'attente d'un navire (en heures) avec x_1 = tirant d'eau et x_2 = nombre de conteneurs à décharger. Une fois le modèle entraîné, $\theta_0, \theta_1, \theta_2$ sont simplement **3 nombres fixes** (par exemple $\theta_0 = 2, \theta_1 = 0.5, \theta_2 = 0.01$).

Étape 2 — les θ_j sont constants pour toutes les lignes. Une fois appris, $\theta_0, \theta_1, \theta_2$ ne changent *plus* : ce sont les mêmes 3 nombres réutilisés pour prédire n'importe quel nouveau navire. Ce qui change d'une ligne (d'un navire) à l'autre, ce sont uniquement les valeurs x_1, x_2 — jamais les θ :

Navire	x_1 (tirant d'eau)	x_2 (conteneurs)	$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2$
A	8	200	$2 + 0.5(8) + 0.01(200) = 8$
B	12	500	$2 + 0.5(12) + 0.01(500) = 13$

Même $\theta_0, \theta_1, \theta_2$ dans les deux lignes — seuls x_1, x_2 changent d'un navire à l'autre. C'est cette propriété (paramètres partagés par toutes les observations) qui permet au modèle de *généraliser* à de nouveaux navires jamais vus.

Étape 3 — généralisation à p features. Pour une observation i avec un vecteur de features $\mathbf{x}_i = (x_{i1}, \dots, x_{ip})$, le modèle prédit :

$$\hat{y}_i = \theta_0 + \theta_1 x_{i1} + \theta_2 x_{i2} + \dots + \theta_p x_{ip} = \boldsymbol{\theta}^\top \mathbf{x}_i$$

Le **double indice** x_{ij} (valeur de la feature j pour l'observation i) distingue bien les deux rôles : i indexe la *ligne* (le navire), j indexe la *feature* (tirant d'eau, nombre de conteneurs, ...). En revanche θ_j n'a **qu'un seul indice** : il ne dépend que de la feature j , jamais de la ligne i — exactement comme dans le tableau ci-dessus.

Remarque de notation : $\boldsymbol{\theta}$ et $\boldsymbol{\beta}$ désignent le même objet (le vecteur de coefficients) — $\boldsymbol{\theta}$ est la notation la plus courante côté Machine Learning, $\boldsymbol{\beta}$ vient de la tradition statistique.

On minimise uniquement par rapport à θ . Les données d'entraînement x_i, y_i sont *fixes* (ce sont des constantes du problème une fois le dataset choisi) — seuls les θ_j sont ajustés pour rendre l'erreur $J(\theta)$ la plus petite possible :

$$\hat{\theta} = \arg \min_{\theta} J(\theta)$$

C'est pour cela que la mise à jour du gradient (plus bas) s'écrit séparément pour chaque θ_j , avec sa propre dérivée partielle $\frac{\partial J}{\partial \theta_j}$: elle mesure combien J changerait si l'on bougeait légèrement *seulement* ce coefficient, tous les autres θ et toutes les données restant fixes. En pratique, tous les θ_j sont mis à jour simultanément à chaque itération.

Fonction de coût (Ordinary Least Squares). On cherche les paramètres qui minimisent la moyenne des carrés des résidus (*Mean Squared Error*) sur les m observations d'entraînement :

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

Pourquoi le carré ? Le carré permet :

- d'éviter que les erreurs positives et négatives s'annulent ;
- de pénaliser fortement les grandes erreurs ;
- d'obtenir une fonction facilement optimisable par dérivation.

Le facteur 2 (qui apparaît dans certaines écritures de J , par ex. $\frac{1}{2m} \sum (\hat{y}_i - y_i)^2$) est principalement une convention mathématique qui facilite la dérivée : il s'annule avec le 2 provenant de la dérivée du carré.

Pourquoi m ? On veut calculer une moyenne des erreurs — diviser par m (le nombre d'observations) rend le coût comparable d'un dataset à l'autre, indépendamment du nombre d'exemples.

Lien avec Gradient Descent. On utilise souvent la descente de gradient pour trouver les paramètres θ qui minimisent cette fonction :

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$$

où α est le *learning rate*.

Hypothèses statistiques du modèle (utile si on te pousse sur la théorie) : linéarité de la relation, indépendance des résidus, homoscedasticité (variance des résidus constante), normalité des résidus, absence de multicollinéarité forte.

- **Type de problème** : régression (target continu).
- **Cas d'usage** : prédire le temps d'attente d'un navire en heures, prédire un montant, une quantité, un prix.
- **Limites** :
 - Suppose une relation *linéaire* entre features et target — capte mal les relations complexes/non linéaires.
 - Sensible aux *outliers* (valeurs extrêmes qui tirent la droite).
 - Sensible à la *colinéarité* entre features (deux features très corrélées entre elles perturbent l'interprétation des coefficients).
- **Avantage clé à mentionner** : très interprétable — chaque coefficient θ_j indique l'effet direct et quantifiable de la feature x_j sur le target, ce qui est précieux pour justifier une décision auprès d'une Direction.

2.2 Logistic Regression

Principe : malgré son nom, c'est un modèle de *classification*, pas de régression. Il prédit une *probabilité* (entre 0 et 1) d'appartenir à une classe, en appliquant une fonction sigmoïde à une combinaison linéaire des features. On fixe ensuite un seuil (souvent 0,5) pour décider de la classe finale : si probabilité $\geq 0,5 \rightarrow$ classe 1, sinon classe 0.

Formalisation mathématique

Fonction sigmoïde (logistique). On combine linéairement les features comme en régression linéaire, $z_i = \beta^\top \mathbf{x}_i + \beta_0$, puis on écrase ce score réel dans l'intervalle $[0, 1]$ via :

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \Rightarrow \quad P(y_i = 1 \mid \mathbf{x}_i) = \hat{p}_i = \sigma(z_i) = \frac{1}{1 + e^{-(\beta^\top \mathbf{x}_i + \beta_0)}}$$

Log-odds (logit). La régression logistique modélise en fait le logarithme du rapport de cotes comme une fonction linéaire des features, ce qui justifie le nom "logistic" :

$$\text{logit}(\hat{p}_i) = \ln\left(\frac{\hat{p}_i}{1 - \hat{p}_i}\right) = \beta^\top \mathbf{x}_i + \beta_0$$

Un coefficient β_j s'interprète donc comme l'effet de x_j sur le *log-odds* : e^{β_j} est le facteur multiplicatif appliqué au rapport de cotes quand x_j augmente de 1 unité.

Fonction de coût (Binary Cross-Entropy / Log-Loss). On ne peut pas utiliser la somme des carrés (non convexe ici) : on maximise la vraisemblance des observations, ce qui revient à minimiser la *log-loss* sur les n observations ($y_i \in \{0, 1\}$) :

$$J(\beta, \beta_0) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \ln(\hat{p}_i) + (1 - y_i) \ln(1 - \hat{p}_i) \right]$$

Cette fonction est convexe en β , ce qui garantit un minimum global unique.

Optimisation. Contrairement à la régression linéaire, il n'y a pas de solution analytique fermée : on optimise J par descente de gradient (ou variantes comme Newton-Raphson / IRLS) :

$$\beta_j \leftarrow \beta_j - \alpha \frac{\partial J}{\partial \beta_j} \quad \text{avec} \quad \frac{\partial J}{\partial \beta_j} = \frac{1}{n} \sum_{i=1}^n x_{ij} (\hat{p}_i - y_i)$$

Remarque : cette dérivée a exactement la même forme que pour la régression linéaire (prédiction — réel, pondéré par la feature) — c'est une propriété élégante commune à tous les modèles linéaires généralisés (GLM) avec une fonction de lien canonique.

- **Type de problème** : classification (le plus souvent binaire ; extensible au multi-classe).
- **Cas d'usage** : prédire si un navire sera en retard (oui/non), détecter une anomalie ou une fraude, scoring d'un dossier (accepté/refusé).
- **Limites** :
 - Suppose que les classes sont *linéairement séparables* (une frontière de décision simple)
 - performe mal sur des frontières complexes.
 - Sensible aux features non standardisées et aux outliers.
 - Moins performante que Random Forest/XGBoost sur des données avec beaucoup d'interactions entre variables.
- **Avantage clé à mentionner** : tout comme la régression linéaire, elle reste interprétable (coefficients = effet sur le log-odds) et donne directement une probabilité utile pour prioriser des cas (ex. classer les navires par risque de retard décroissant), pas seulement une étiquette binaire.

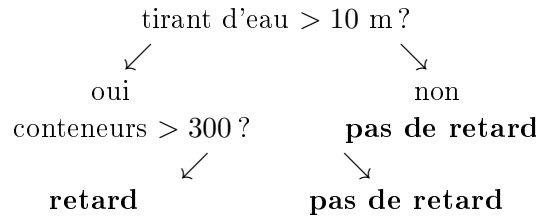
Point à ne pas confondre à l'oral : Linear Regression prédit un *nombre continu* ; Logistic Regression prédit une *probabilité*/catégorie. Les deux sont des modèles linéaires simples, souvent utilisés comme *baseline* avant de tester des modèles plus complexes (Random Forest, XGBoost).

2.3 Decision Tree

Étape 1 — l'intuition. Un arbre de décision pose une suite de questions simples de type "la feature x_j est-elle $\leq$ un seuil s ?" et découpe les données en deux groupes à chaque question,

jusqu'à obtenir des groupes finaux (*feuilles*) les plus "purs" possible (majoritairement une seule classe, ou une valeur de target homogène).

Exemple portuaire : "tirant d'eau > 10 m ?" → si oui, "nombre de conteneurs > 300 ?" → ... jusqu'à la feuille qui donne la prédiction (ex. "retard probable" / "pas de retard").



Étape 2 — comment choisir la meilleure question ? À chaque nœud, l'arbre teste toutes les features et tous les seuils possibles, et choisit celui qui rend les groupes résultants les plus purs. Pour mesurer l'impureté d'un groupe, deux critères principaux :

Indice de Gini (classification), pour un nœud contenant K classes de proportions $p_1, \dots, p_K$:

$$\text{Gini} = 1 - \sum_{k=1}^K p_k^2$$

Gini = 0 si le nœud est parfaitement pur (une seule classe) ; Gini est maximal quand les classes sont mélangées à parts égales.

Entropie (alternative à Gini, issue de la théorie de l'information) :

$$\text{Entropie} = - \sum_{k=1}^K p_k \log_2(p_k)$$

Gain d'information. On choisit la question (feature + seuil) qui *maximise la réduction d'impureté* entre le nœud parent et ses deux enfants (gauche G , droit D), pondérée par la taille de chaque enfant :

$$\text{Gain} = \text{Impureté}_{\text{parent}} - \left(\frac{n_G}{n} \text{Impureté}_G + \frac{n_D}{n} \text{Impureté}_D \right)$$

où n est le nombre d'observations au nœud parent, n_G et n_D le nombre d'observations envoyées respectivement à gauche et à droite.

Cas de la régression : au lieu de Gini/entropie, on minimise la variance (ou la MSE) des valeurs de target dans chaque groupe — même logique de la fonction de coût que pour Linear Regression, appliquée localement à chaque nœud.

Arrêt de la construction. L'arbre s'arrête de grandir quand une condition est atteinte : profondeur maximale (**max_depth**), nombre minimal d'observations par feuille (**min_samples_leaf**), ou gain d'information devenu négligeable.

- **Type de problème** : classification et régression.
- **Cas d'usage** : règles de décision explicables (ex. scoring d'un dossier avec des critères métier lisibles), segmentation de navires par profil de risque.
- **Limites** :
 - **Overfitting très facile** si l'arbre est laissé libre de grandir (il peut isoler chaque observation dans sa propre feuille).
 - **Instable** : une petite variation dans les données d'entraînement peut produire un arbre très différent (forte variance).
 - Frontières de décision "en escalier" (perpendiculaires aux axes), donc moins naturelles que celles d'un modèle linéaire pour des relations lisses.
- **Avantage clé à mentionner** : très interprétable et visualisable (on peut littéralement dessiner l'arbre et le montrer à un métier non technique) ; ne nécessite ni scaling ni normalisation des features.

2.4 Random Forest

Étape 1 — le problème que ça résout. Un seul Decision Tree a une forte variance (instable, sensible à l'overfitting). L'idée du Random Forest : au lieu d'un seul arbre, en construire *beaucoup* (ex. 100, 500) et faire la **moyenne** de leurs prédictions (régression) ou un **vote majoritaire** (classification). Des erreurs individuelles d'arbres différents ont tendance à s'annuler, ce qui réduit la variance globale sans trop augmenter le biais.

Étape 2 — comment rendre les arbres différents entre eux ? Si tous les arbres étaient construits sur exactement les mêmes données avec les mêmes règles, ils seraient identiques et la moyenne n'apporterait rien. Random Forest introduit deux sources d'aléa (technique dite du *bagging* = *Bootstrap Aggregating*) :

- **Bootstrap sur les lignes** : chaque arbre est entraîné sur un échantillon tiré *avec remise* (même taille m que le dataset original, mais certaines lignes apparaissent plusieurs fois, d'autres pas du tout).
- **Sélection aléatoire de features** : à chaque nœud, l'arbre ne considère qu'un sous-ensemble aléatoire des features (souvent $\sqrt{p}$ pour la classification, $p/3$ pour la régression, avec p = nombre total de features) au lieu de toutes les tester — cela décorrèle davantage les arbres entre eux.

Formalisation de l'agrégation. Pour T arbres entraînés $f_1, \dots, f_T$:

$$\hat{y}_{\text{régression}} = \frac{1}{T} \sum_{t=1}^T f_t(\mathbf{x}) \qquad \hat{y}_{\text{classification}} = \text{mode}(f_1(\mathbf{x}), \dots, f_T(\mathbf{x}))$$

c'est-à-dire la moyenne des T prédictions numériques, ou la classe la plus votée parmi les T arbres.

Pourquoi ça marche (intuition de la réduction de variance). Si chaque arbre a une variance σ^2 et que les erreurs des arbres sont peu corrélées, la variance de la moyenne de T arbres tend vers σ^2/T — d'où l'intérêt de maximiser T (dans une limite raisonnable) et de décorrélérer les arbres (bootstrap + sélection aléatoire de features).

Feature importance (bonus utile à mentionner) : Random Forest fournit naturellement un score d'importance par feature, en mesurant de combien chaque feature réduit l'impureté (Gini/variance) en moyenne sur tous les arbres et tous les nœuds où elle est utilisée — utile pour expliquer un modèle à une Direction.

- **Type de problème** : classification et régression.
- **Cas d'usage** : la plupart des problèmes tabulaires "par défaut" (scoring, prévision, détection d'anomalie) — bon compromis performance/simplicité de mise en œuvre.
- **Limites** :
 - Moins interprétable qu'un seul Decision Tree (on ne peut plus "dessiner" la décision)
 - on s'appuie sur la feature importance en remplacement.
 - Plus coûteux en mémoire/temps de calcul qu'un modèle linéaire ou qu'un seul arbre (il faut stocker et faire tourner T arbres).
 - Peut rester moins performant que du Gradient Boosting/XGBoost sur des données très structurées, car les arbres sont construits indépendamment (pas de correction séquentielle des erreurs).
- **Avantage clé à mentionner** : robuste par défaut (peu de tuning nécessaire), gère bien les valeurs manquantes/outliers, réduit fortement l'overfitting par rapport à un arbre seul
 - c'est souvent le premier modèle "sérieux" à essayer après une baseline linéaire.

Question type à savoir répondre à l'oral : « *Pourquoi choisir Random Forest plutôt que Logistic Regression ?* » → Random Forest capte des relations non linéaires et des interactions complexes entre features sans les spécifier à la main, gère nativement les variables catégorielles et les valeurs manquantes, et est moins sensible aux outliers — au prix d'une interprétabilité directe des coefficients (compensée par la feature importance). Logistic Regression reste préférable quand

l'interprétabilité fine (impact exact et signé de chaque variable) est exigée, ou quand le volume de données est trop faible pour justifier un modèle complexe.

2.5 XGBoost

2.6 K-Means

2.7 PCA

3 Évaluation ML

3.1 Classification — Confusion Matrix

3.2 Classification — Métriques

3.3 Régression — Métriques

3.4 Questions types

4 ML avancé (Jour 3)

4.1 Preprocessing

4.2 Feature engineering et feature selection

4.3 Data leakage

4.4 Imbalanced dataset et SMOTE

4.5 Cross-validation

4.6 Hyperparameter tuning

4.7 Gradient Boosting et Ensemble Learning

4.8 Questions ouvertes